Juniper

CLI coding assistant | Project report

Group Project 2 - 15 October 2026

1. System and design decisions

Juniper accepts a natural-language coding task in a terminal. An agent loop asks an LLM what to do, executes validated MCP tool calls, returns their results to the model, and streams its answer. It can inspect and edit files, run Python scripts/tests, and retrieve local or external documentation. Groq and Ollama share the provider interface; the model comparison uses two different Groq-hosted LLMs.

Design decision	Reason and implemented behavior
Separate providers from the agent	A shared streaming interface lets the same agent and tools run with Groq or Ollama. Model comparisons therefore reuse the execution pipeline.
Discover tools through MCP	The required filesystem, hosted DeepWiki, and custom RAG servers are joined by a Python-check server. All tools are dynamically discovered through MCP.
Validate before execution	The dispatcher checks names and argument schemas. Confirmation is the default. Python checks validate the target path, use no shell, omit credential environment variables, and bound duration/output; executed code is trusted, not OS-sandboxed.
Keep retrieval local and traceable	Chroma stores document embeddings. Returned snippets retain source, page, and chunk references. Cross-encoder reranking improves ordering within retrieved candidates.
Bound cloud requests and recovery	Adapters compact history, pace requests, and expose bounded retries. Evaluation checkpoints allow later resume. Full local history is retained; account limits remain external.

Evidence status

This report includes the completed baseline and intermediate diagnostic runs. The latest reliability fixes have regression coverage, but a complete live retest of the final policy is still pending after Groq quota exhaustion. Passing local tests is not proof of successful autonomous coding.

Report guide: model comparison p. 2; measured RAG evaluation p. 3; reflection/evidence p. 4; original planning architecture p. 5; updated architecture p. 6; state and three sequence diagrams pp. 7-10.

2. Two LLMs on the same coding tasks

Models: **openai/gpt-oss-20b** and **openai/gpt-oss-120b**, both through Groq. The baseline used four identical task definitions, three repetitions per model, fresh conversations, and isolated copies of the same initial files: 24 scored attempts. Only workspace paths differed in task prompts. All runs used the production agent, all three MCP servers (18 discovered tools), auto execution, up to eight iterations, and a 180-second task deadline. Baseline generation settings were SDK defaults.

A code-check pass means saved code met the evaluator's behavioral checks. An end-to-end pass additionally requires successful read/edit tools, required retrieval tools for the documentation task, normal completion, and no terminal agent error. Checks run outside the model's accessible fixture root.

Identical task	20B end-to-end	120B end-to-end
Fix mean calculation; raise ValueError for empty input	1 / 3	1 / 3
Extract unique ASCII words in first-seen order	3 / 3	0 / 3
Refactor sales totals into a shared helper	0 / 3	0 / 3
Implement UTF-8 greeting writer using retrieved docs	0 / 3	0 / 3
Total end-to-end passes	4 / 12 (33.3%)	1 / 12 (8.3%)
Total code-check passes	4 / 12	3 / 12

Insights from the baseline

20B completed the ASCII-word feature on all three attempts; 120B repeatedly requested unqualified tool names. Conversely, 120B produced two correct sales refactors, but subsequent requests exceeded the account's 8,000-token-per-minute allowance, preventing end-to-end passes. Both models fixed the mean only on their third attempt. Tool use, recovery, and quota constraints strongly affected outcomes; model size alone did not predict success.

Subsequent diagnostics and current limits

Two intermediate eight-run pilots scored **3/8** and **2/8**. The second had zero dispatched tool errors, but still failed on limits, Windows indentation, and ASCII/Unicode logic. A later free-tier batch with pacing, checkpoints, and syntax checks passed the bug-fix task with **both models**. The next slot hit the daily cap before tool use, and the runner paused: **2/3 attempted, 2/8 planned**, with five slots unattempted. This remains a partial retest, not a clean overall improvement estimate or general model ranking.

Sources: evaluations/model_comparison/results/2026-10-15/, results/2026-10-15-fixes/, and results/2026-10-15-free-tier/. Each retains its own report, traces, source files, and metadata.

3. Advanced RAG and insights

The chosen technique is **cross-encoder reranking**. The corpus contains FastMCP documentation and Python, JavaScript, and Java PDFs. Text is chunked and embedded with **all-MiniLM-L6-v2**; Chroma indexes 10,473 chunks. At query time, cosine search finds candidates, then **cross-encoder/ms-marco-MiniLM-L6-v2** jointly scores each query/passage pair. The server returns original snippets and references in the new order.

Why this technique

A semantically related passage can rank highly without answering the question. Reranking adds a query-aware relevance step while reusing the existing vectors. The MCP tool defaults to reranking; cosine mode remains available as the baseline. Cosine similarity and cross-encoder logits are distinct scores, and neither is a confidence percentage.

Paired experiment: relevance, answers, and grounding

Ten fixed questions cover all four documents: eight intended answerable questions, a Path.write_text coverage probe, and an unrelated negative control. Both modes use the same production stdio MCP server and index, top_k=5, and no source filter. Reranking scores 20 cosine candidates. Generation uses the same Groq 20B adapter, evidence-only prompt, temperature 0, and SDK-default text-only reasoning; only the five supplied passages differ. Mode order alternates. This isolates context quality, not autonomous tool selection.

Metric	Cosine baseline	Cross-encoder
Direct evidence at rank 1 (8 queries)	25.0%	50.0%
Precision@5 (directly useful passages)	25.0%	35.0%
Mean reciprocal rank@5	0.400	0.688
Correct reference facts (5 answer pairs)	8/10	10/10
Fully grounded answers	2/5	4/5
Median warm retrieval time (10 queries)	0.021 s	0.430 s

What we learned

Reranking brought a complete FastMCP tool example into the final context; the baseline answer invented unsupported decorator arguments. Python break moved above a JavaScript passage. Gains were uneven: the reranked Java finally result was JavaScript, and the Python continue context lost a useful Python definition. Neither mode retrieved the required Path.write_text API detail. Source scope and corpus coverage remain important.

Groq's daily limit stopped generation after five complete pairs (10/20 planned answers); remaining answers, including controls, were not tested. All 100 final passages and 200 recorded candidates matched prepared text/metadata. A single assessor graded passages: 2 directly supports a reference fact in the requested language, 1 is related background, 0 is irrelevant/wrong-language. Precision/MRR count grade 2; answer correctness and full claim support are separate. Small samples, no blinded second rater, one generation per condition, and approximate search limit generalization. Warm timings exclude model loading and are not a controlled latency benchmark.

Reproduce: evaluations/rag_evaluation/README.md and results/2026-10-15/. Method reference: NirDiamant RAG_Techniques - Reranking (accessed 15 October 2026). Answer comparisons do not establish improved autonomous coding pass rates.

4. Reflection and supporting evidence

What worked

The provider-neutral loop connected the required tools and retained code/trace evidence. Several coding attempts saved correct implementations. The measured RAG comparison shows useful changes in passage ordering and answer grounding. Final regression checks passed 249 tests; strict type checking passed for 65 files.

What did not work

Incorrect tool names, null arguments, malformed output, long histories, and quotas interrupted tasks. Windows line-ending mismatches could damage indentation. Tool success did not imply correct code: a later 120B feature mishandled Unicode when enforcing ASCII words. Retrieval sometimes returned related but insufficient evidence; scanned PDF pages limited corpus coverage.

Changes made and what we would improve

Implemented fixes include tool aliases, null cleanup, bounded recovery, context compaction, Windows-edit normalization, and the correct RAG interpreter. Free-tier controls pace requests and checkpoint for resume. Python edits receive policy-controlled reads and syntax checks; failures feed back for repair and block normal completion. Syntax checks do not prove behavior.

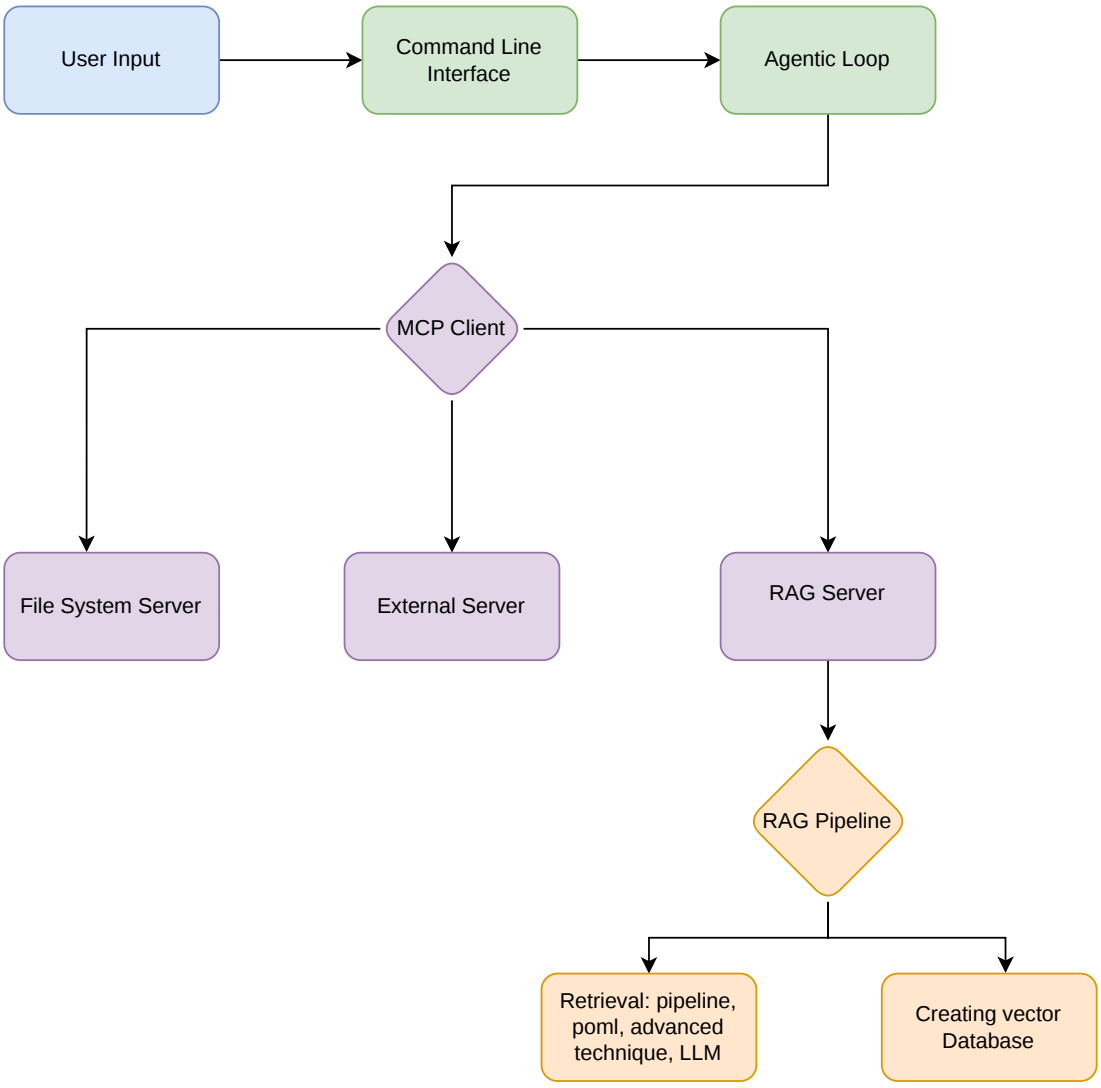
The final deliverables add a separate MCP tool for trusted Python scripts or pytest, with visible approval, exit status, timeouts, and bounded returned output. It is a building block for behavioral checks, not proof of adequate test coverage. The measured RAG comparison now evaluates relevance, answer facts, and citation grounding. Next we would complete the current-policy coding retest, improve corpus coverage/source filtering, and use more queries, repeated generations, and independent blinded raters. Groq quotas and outages remain outside the project.

Evidence index (paths relative to repository root)

Evidence	Location
24-run baseline and saved code	evaluations/model_comparison/results/2026-10-15/
Fixes, 16 diagnostic runs, final-policy hashes	evaluations/model_comparison/results/2026-10-15-fixes/
Measured RAG comparison and answer judgments	evaluations/rag_evaluation/README.md; results/2026-10-15/
Free-tier controls and syntax validation	evaluations/model_comparison/FREE_TIER_TESTING_NOTES.md
Original planning architecture (p. 5)	init-architecture.drawio2.pdf
Current structure and configuration (p. 6)	docs/architecture.md; config/mcp.json; src/coding_assistant/; rag_server/

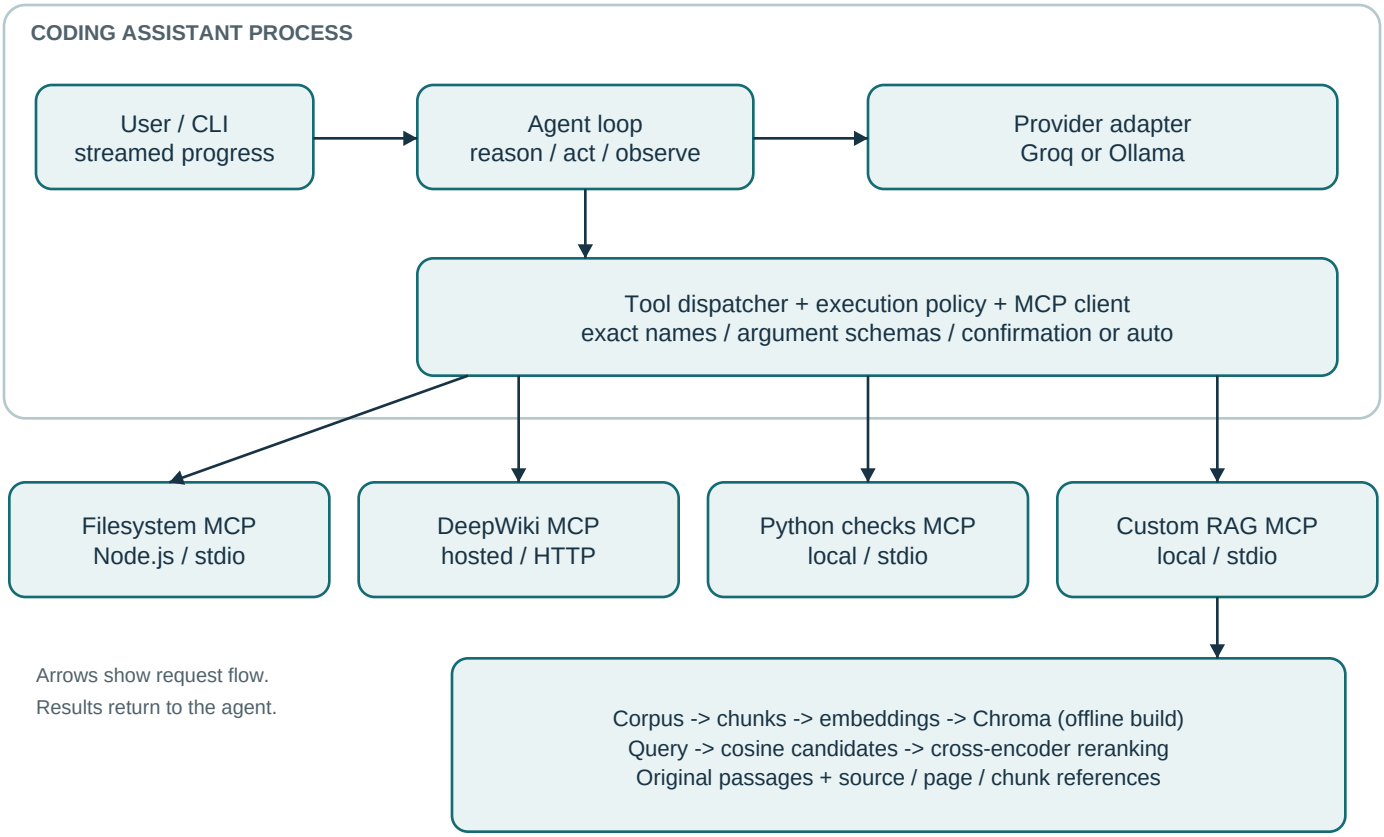
The coding baseline evaluated commit 6e4735d; later fixes have retained source hashes. The RAG experiment records base commit 031edeb plus executed source hashes. Later command-tool additions were not part of those coding trials. The video demo remains a separate deliverable for the team. Page 5 reproduces the original planning PDF unchanged.

Initial Architecture



5. Updated architecture

The original planning diagram appears on page 5. This diagram shows the final system, including reliability changes and the Python-check server. It preserves the planned CLI, agent, MCP client, and three required server roles.



What changed from the plan, and why

Model boundary made explicit. Groq/Ollama adapters sit outside the agent's coordination logic so models can change without replacing the tool pipeline. The planning diagram did not show this boundary.

Execution checks added explicitly. Discovered tool names, schemas, and confirmation policy sit before MCP calls. Provider-side aliases, context budgeting, and retries address the failures observed during evaluation.

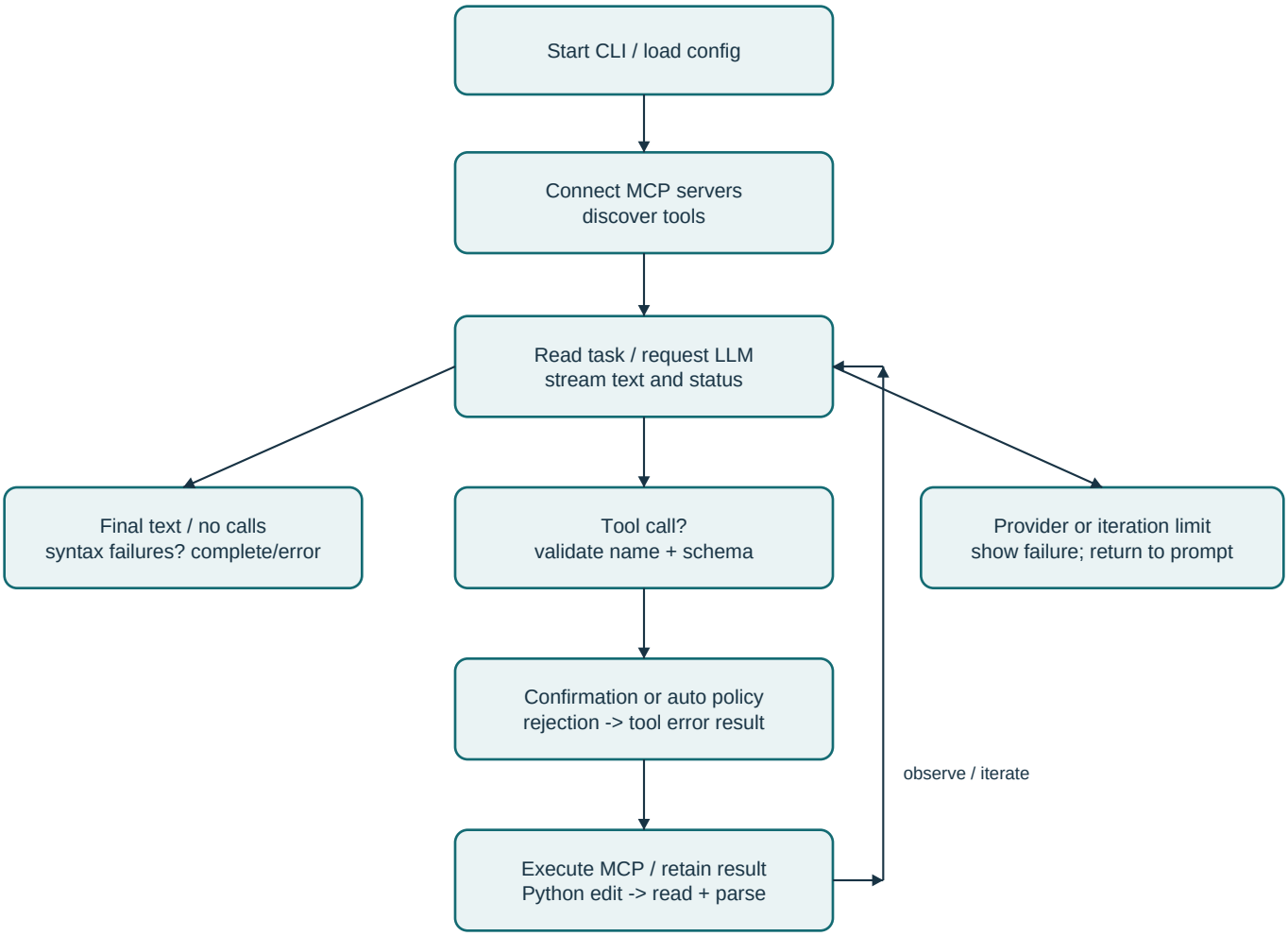
RAG design made concrete. The plan's generic retrieval/vector-database branches became an offline ingestion/index build and a query-time cosine-plus-cross-encoder pipeline. Answer generation remains in the main agent/provider; the RAG server returns evidence. Chroma and reranking were chosen to reuse vectors and improve passage ordering.

Deployment boundaries clarified. Filesystem and RAG are local stdio processes; DeepWiki is hosted over HTTP. The RAG executable follows the running Python interpreter to avoid missing dependencies. These refinements explain how the original architecture became executable.

Command execution added. A separate local Python-check MCP server runs trusted scripts/tests with per-call policy, target validation, time limits, credential-environment omission, and bounded returned output. This fills the assignment's command-execution capability without adding a shell interface. It is not an OS sandbox.

6. Task state diagram

The state flow combines user input, model decisions, validation, approval, MCP execution, feedback, and completion. See docs/diagrams.md for editable Mermaid sources and error branches.



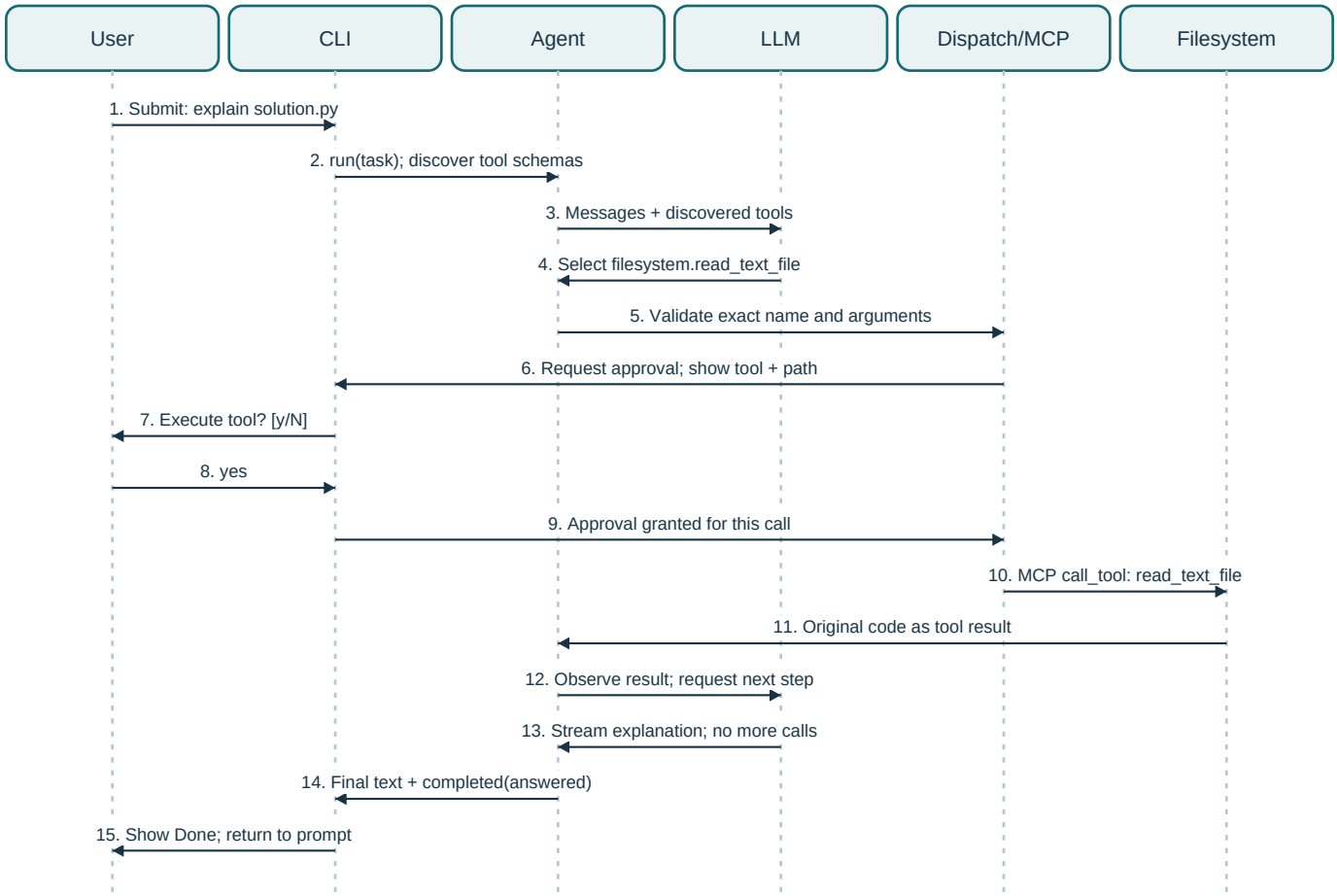
Completed or failed task -> next prompt. /exit or interruption -> close clients and exit.

Invalid calls return errors without execution. Groq retry/pacing is bounded inside its adapter.

The CLI can run many tasks in one session. One-shot mode exits after its task. All clients close on exit. The LLM sees full tool results through the agent; the Groq adapter may compact the request view. Python syntax feedback is added after writes, while optional behavioral execution is a distinct checks.run_python_check tool call.

Inspect existing code

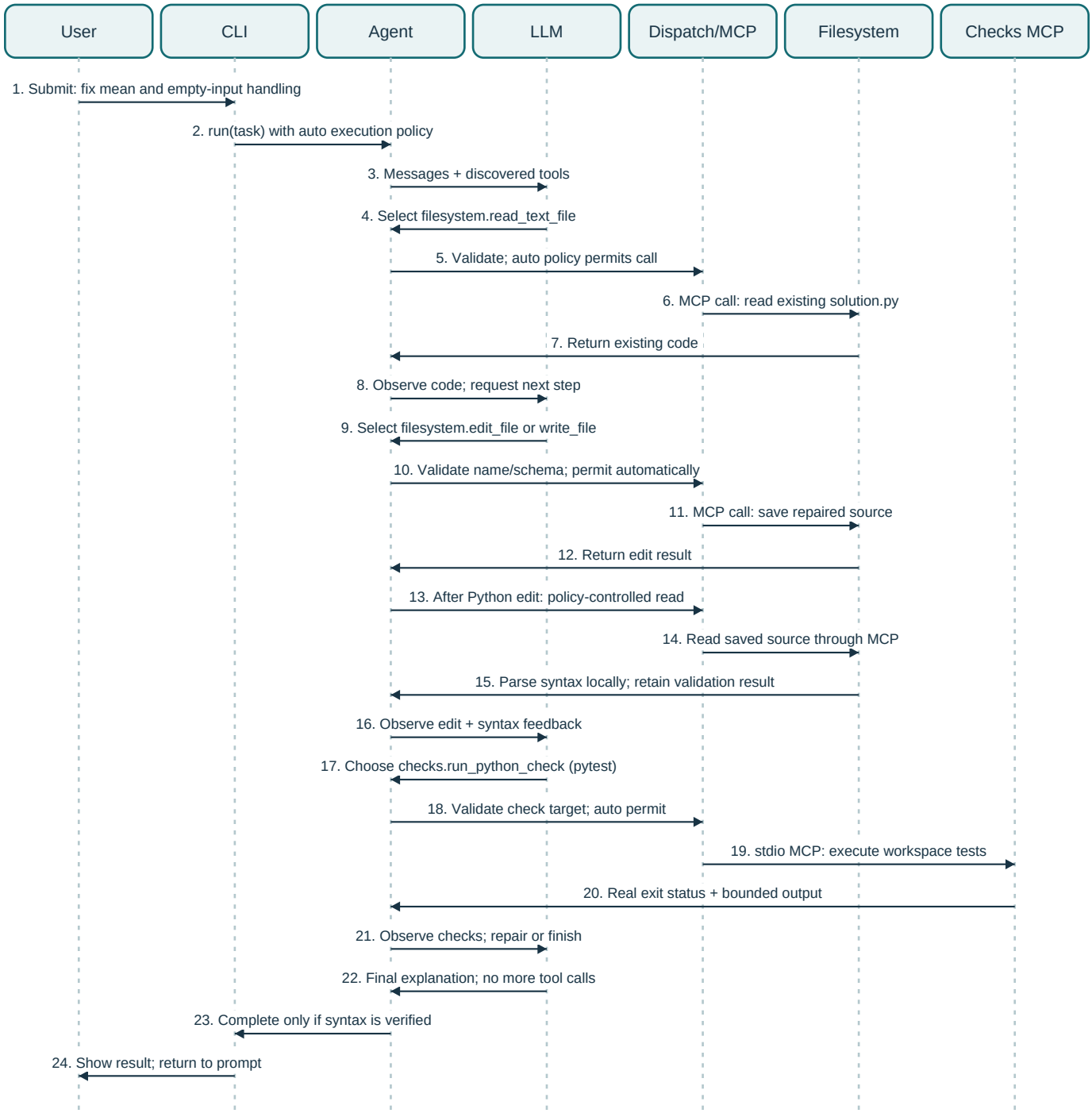
Confirmation mode | operation 1



If approval is rejected, no filesystem call occurs. The rejection is returned to the LLM as a tool error, so it can explain or revise its plan.

Repair a Python bug

Automatic mode | operation 2

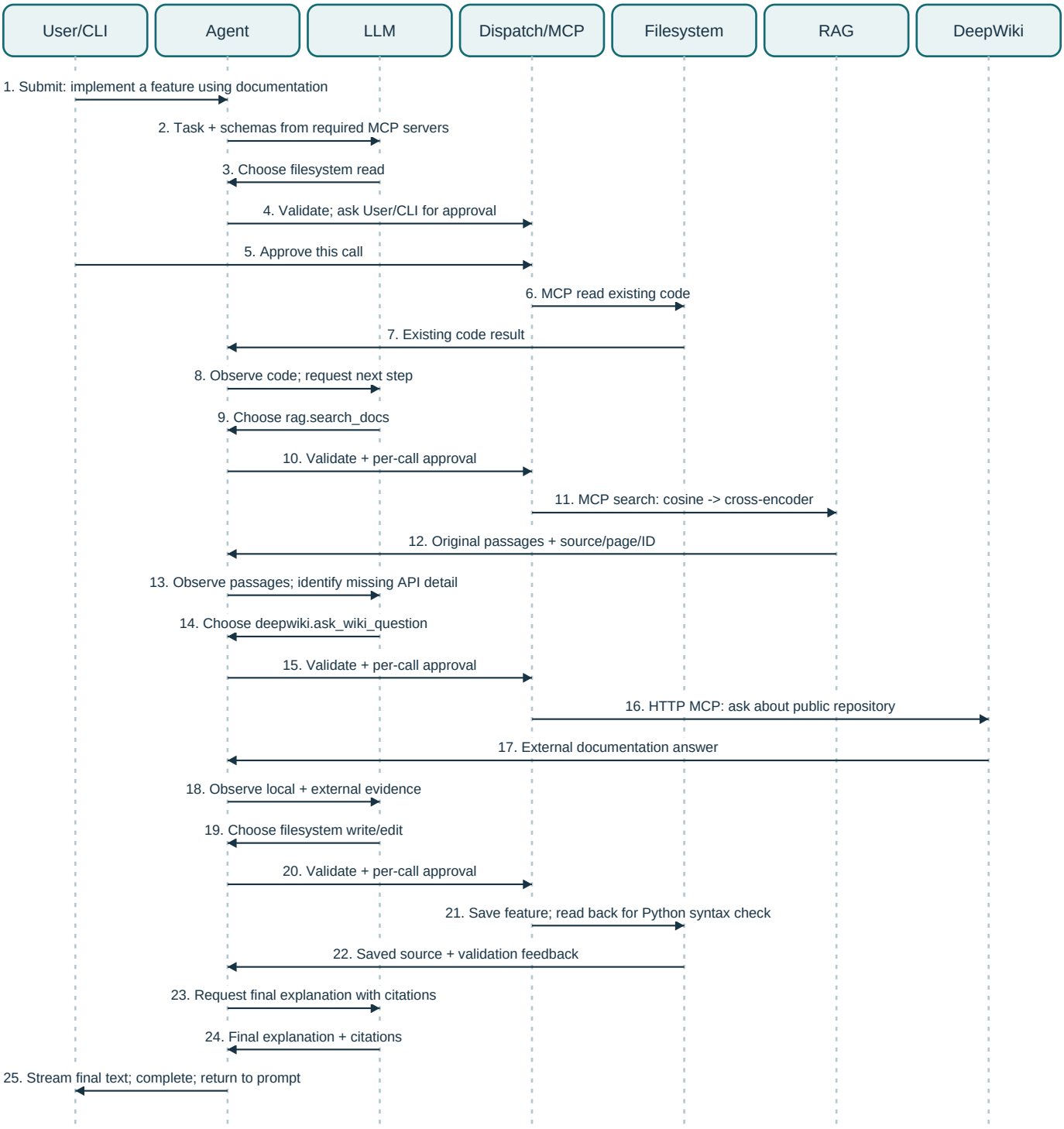


Syntax errors feed back for repair and block normal completion until resolved. The model can request the separate Python-check tool to run behavioral tests; those tests establish only what they check. Historical coding evaluation checks ran separately outside the model's writable fixture.

The optional Python-check call shown here was added after the recorded coding comparison. Test completeness and assertions determine what a passing check means.

Implement a documented feature

Confirmation mode | operation 3



Each valid call, including syntax-check reads, requests approval in confirmation mode. The model chooses tools and their order; this is an implementation walkthrough, not a claim that every live task succeeds. If local evidence is insufficient, retrieve another source or acknowledge the gap.